

Testing & evaluating an innovative solution

You built the thing. It runs. Now the hardest question in the whole project: does it actually work, and is it actually any good? Those are two different questions — and mixing them up is the single most common way to lose marks in this outcome.

What this key knowledge covers

This summary distils everything you need for **KK10 — Testing & evaluating an innovative solution** in Innovative Solutions. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

□ The scenario: GrazeGo

Back in KK06 you met **GrazeGo** — four Year 11 students (Aiden, Priya, Mason and Lily) building a canteen **pre-order app** so the lunch rush stops being a 200-person crush. They surveyed users, designed it, and built a working prototype. The app lets a student pick a meal before recess, pay, and collect from a fast lane at lunch.

It's *built*. The demo works on Priya's phone. The team is ready to call it done. This module is about everything that has to happen **before** they're allowed to.

Testing ≠ Evaluating

VCAA examiners ask this every year, and students throw away marks by treating the two words as synonyms. They are not. They happen at **different times**, ask **different questions**, and compare against **different things**.

Does it work? Checking that each part of the solution does what it's *supposed* to do — that it meets the **requirements / specifications**. Done **during and at the end of development**, usually with a **testing table**.

How well does it do the job? Judging how *efficient* and *effective* the solution is in real use, against **evaluation criteria** written back in the **design** stage. Done **after the solution has been in use for a period of time**.

Testing is tasting a slice straight out of the oven: is it cooked, is it the right size, did the cheese melt? You're checking it matches the recipe (the requirements). **Evaluating** is asking your friend a week later: "*Would you order that again? Was it actually good value? Did it fill you up?*" That's judging how well it met their real need — and you can only answer it after they've



actually lived with it.

The timeline below shows the two living in different places. Drag the scrubber to move through GrazeGo's project. Notice **testing** sits at the tail of *development*, while **evaluation** only switches on *after* the app has been used in the canteen for a few weeks.

"The team **evaluated** the app by checking each button worked." — **No**. Checking buttons work is **testing**. Evaluation would be judging whether GrazeGo actually reduced the lunch queue and how efficiently students could order. Swap those two words and you've answered the wrong question.

User testing

You hand the solution to **real intended users** and watch them use it — without coaching them. Their stumbles, confusion and feedback reveal problems the makers are blind to, because the makers already know how it's "supposed" to work.

How GrazeGo runs a user test

They grab **five Year 8 students** (real canteen users, not friends from their own class) and give each a realistic **task**: "*Order a chicken wrap and an apple juice, and pay, before the timer hits 60 seconds.*" Then the team does three things:

Watch silently. Where do they hesitate? What do they tap by mistake? Did they finish the task at all?

Ask users to narrate their thoughts: "*I can't find the pay button...*" This exposes **why** they're stuck, not just that they are.

A short post-task survey or interview: what was confusing, what was easy, would they use it for real?

When GrazeGo tested, **4 of 5** students missed the "Pay" button because it was the same grey as the background. The team *knew* where it was, so they'd never noticed. That's a **usability** problem only a real user would surface — and it directly improves the solution.

Ever yank a door that says PUSH? You're not stupid — the door is badly designed. The designer knew how it worked, so they couldn't see the problem. User testing is hiring strangers to walk into your doors so you find out which ones are confusing **before** you ship them.

Strengths & limitations

Reveals **real-world usability** issues; uncovers needs the team can't see; feedback comes from the actual audience; builds confidence the solution suits its users.

Time-consuming to organise; users may be polite rather than honest; a tiny sample may not represent everyone; users spot *symptoms* but rarely the technical *cause*.

Expert review

Someone with **specialist knowledge** — a professional developer, a UX/accessibility expert, an experienced teacher — inspects the solution against **best practice and established standards**. They catch things ordinary users never could.

Who's an "expert" for GrazeGo?

Their teacher invites **Mr Tan**, a software developer who builds payment apps for a living. He spends 20 minutes with GrazeGo and writes up findings against professional standards:

User testing tells you *where people struggle*. **Expert review** tells you *what's wrong underneath* — security, performance, accessibility, code quality. Strong projects use **both**, because each sees what the other misses.

Strengths & limitations

Catches deep technical, security and accessibility issues; fast (no need to recruit a crowd); measures against recognised standards; spots problems before they reach users.

Experts can be hard to find or costly; one expert's opinion may be biased; they may miss **real-user** behaviour because they're not the target audience; advice can be over-technical for a school project.

The testing table

A **testing table** is how you *prove* you tested — methodically, not just "I clicked around and it seemed fine." Each row is one test with a clear **expected** result and a recorded **actual** result, so anyone can see exactly what passed, what failed, and what you did about it.

Anatomy of a row

A complete row answers six questions. Click any hidden cell below to reveal the worked GrazeGo example.

A test with no **expected** result isn't a test — it's just poking the app. You only know something *failed* if you decided beforehand what success looks like. Row 2 above is the gold: a **fail**, a clear **cause**, a **correction**, and (critically) a note to **re-test**. Examiners reward the full loop.

What makes test data good?

Right at the edge of a limit

Input that should be rejected

Evaluation: efficiency & effectiveness

Testing is over. The app works. Weeks later, GrazeGo asks the real question: **was it worth it?** Evaluation judges the solution against the **evaluation criteria written in the design stage**, and almost always splits into two ideas you must keep separate.

Does it do the job well? How completely and accurately it meets the user's need. Measures: completeness, accuracy, readability, usability, accessibility, clarity, error-tolerance, relevance.

Does it do it with little waste? How well it uses time, cost and effort. Measures: speed, time to complete a task, processing/data cost, money saved, labour saved.

A solution can be **effective but not efficient** (GrazeGo cuts the queue beautifully, but the app takes 40 seconds to load each order). Or **efficient but not effective** (it's lightning fast, but students still queue because only 10% bothered to use it). You must judge them **separately**.

Plot GrazeGo on the map below. Drag the two sliders to set how effective and how efficient it turned out — the marker moves into a quadrant and tells the team what to do next.

If the question says "*after the app had been used for three weeks...*" it's asking you to **evaluate**. If it says "*check that the order total calculates correctly...*" it's asking you to **test**. The verb the question uses tells you which world you're in.

Sort it yourself

Drag each activity into the right bucket. Is it a **testing** activity, or an **evaluating** activity? Get them all green to prove you've nailed the distinction.

Command words

The verb in an exam question tells you exactly how much to write. Tap each card to see what examiners want — and how it links to this topic.

Six exam traps

These are the mistakes that quietly cost marks every year. Tap to see the fix.

They are assessed as **different concepts**. Testing = checking against requirements (does it work). Evaluating = judging efficiency & effectiveness against criteria (how well, after use).

Without a predicted result, there's no way to say a test passed or failed — so it isn't really a test.

Testing only valid, typical data misses where solutions actually break.

"It works really fast" is **efficiency**; "it solves the user's problem fully" is **effectiveness**. They're separate.

Vague claims earn nothing. Examiners want a **technique named** (user testing / expert review) and **how it was carried out**.

You can't evaluate fairly against criteria you invent afterwards. The **evaluation criteria are written in the design stage**, before development.

P-E-L writing trainer

High-scoring extended answers follow **Point → Evidence → Link**. Write a response to the prompt below and watch the live meters detect whether you've hit all three — plus the key terms examiners reward.

Prompt (4 marks): The GrazeGo team is deciding how to check their app before launch. *Recommend* a combination of **user testing** and **expert review**, and *justify* why using both is better than using one alone.

VCE-style practice questions

Five questions in the real VCAA style. Try each on paper first, then reveal a **weak** answer, a **strong** answer and the **marking guide**. Self-mark to fill your score ring.

Distinguish between **testing** and **evaluating** an innovative solution.

- 1 mark — testing = checks it works / against requirements (during development).
- 1 mark — evaluating = judges efficiency & effectiveness / against criteria (after use).

Describe how the GrazeGo team could carry out **user testing** on their canteen pre-order app.

- 1 mark — uses real/intended users given a realistic task.

- 1 mark — observe / think-aloud during the task.
- 1 mark — collect feedback and use it to improve the solution.

Compare **user testing** with **expert review** as techniques for testing an innovative solution. Refer to one strength of each.

- 1 mark — a genuine similarity (both find problems / improve the solution).
- 1 mark — user testing described (real users, tasks).
- 1 mark — expert review described (specialist, standards).
- 1 mark — a clear strength stated for each.

GrazeGo must stop orders after the 11:00 am cut-off. Construct **one row** of a testing table to test this, including a suitable piece of **test data**, the **expected result**, and explain why your chosen test data is appropriate.

Key terms

Term	Meaning
Expert finding	Why a user wouldn't catch it
Payment data isn't encrypted in transit	It "works" for the user, but it's a serious security flaw they can't see.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Distinguish

Between testing and evaluating an innovative solution.

✗ A weak answer

Testing and evaluating both check if the solution is good and works properly.

✓ A strong answer

Testing checks whether the solution works as intended against its requirements, done during development (e.g. with a testing table). **Evaluating** judges how efficient and effective the solution is against evaluation criteria, after it has been in use.

How the marks are awarded

- 1 mark — testing = checks it works / against requirements (during development).
- 1 mark — evaluating = judges efficiency & effectiveness / against criteria (after use).

Q2. Describe

How the GrazeGo team could carry out user testing on their canteen pre-order app. Context: GrazeGo lets students pre-order and pay for canteen food to skip the lunch queue.

✗ A weak answer

They would let people use the app and ask if they liked it.

✓ A strong answer

The team would recruit **real intended users** (e.g. students who buy canteen food) and give each a **realistic task**, such as ordering and paying for a meal. They would **observe** the users without helping, ask them to **think aloud** to reveal points of confusion, then **gather feedback** via a short survey or interview. The problems found (e.g. a hard-to-find Pay button) are used to improve the app.

How the marks are awarded

- 1 mark — uses real/intended users given a realistic task.
- 1 mark — observe / think-aloud during the task.
- 1 mark — collect feedback and use it to improve the solution.

Q3. Compare

User testing with expert review as techniques for testing an innovative solution. Refer to one strength of each.

✗ A weak answer

User testing uses users and expert review uses an expert. Both are good ways to test.

✓ A strong answer

Similarity: both techniques aim to find problems in the solution before it is finalised.

Difference: user testing uses **real intended users** completing tasks, so its strength is exposing **real-world usability** issues the makers can't see. Expert review uses a **specialist** checking against standards, so its strength is catching deep **technical, security or accessibility** flaws that ordinary users would never notice.

How the marks are awarded

- 1 mark — a genuine similarity (both find problems / improve the solution).
- 1 mark — user testing described (real users, tasks).
- 1 mark — expert review described (specialist, standards).
- 1 mark — a clear strength stated for each.

Q4. Explain

GrazeGo must stop orders after the 11:00 am cut-off. Construct one row of a testing table to test this, including a suitable piece of test data, the expected result, and explain why your chosen test data is appropriate.

✗ A weak answer

Test: try to order. Expected: it stops you. I picked it because it tests ordering.

✓ A strong answer

What is tested Test data Expected Pass/Fail Orders rejected after cut-off Submit order at 11:01 am "Ordering closed" message; no order created test it **Why this test data is appropriate:** 11:01 is **invalid/abnormal** data just past the boundary, so it directly checks the rejection rule rather than the normal case. Pairing it with a boundary test at exactly 11:00 would make the test set even stronger.

How the marks are awarded

- 1 mark — relevant "what is tested" tied to the cut-off rule.
- 1 mark — appropriate invalid/boundary test data (a time after 11:00).
- 1 mark — clear expected result (rejected / message shown).
- 1 mark — justification that the data is invalid/boundary, testing the rule not the happy path.